

Problem 25 $\Rightarrow$ Given two trees, give an algorithm for checking whether they are mirrors of each other.

Solution $\Rightarrow$ By checking root values are equal. ~~the~~ Recursively check left subtree of one tree is a mirror of subtree of the other.

Algorithm

1. If Both root 1 and root 2 are null, return true.
2. If one is null and the other is not, return false.
3. If $\text{root1.data} \neq \text{root2.data}$, return false.
4. Recursively check
 - Are Mirror (root1.left , root2.right)
 - Are Mirror (root1.right , root2.left)
5. Return true only if Both recursive calls return true.

Code

```
public static boolean areMirror(TreeNode root1, TreeNode root2) {
    // both empty
    if (root1 == null && root2 == null)
        return true;

    // one empty, one not
    if (root1 == null || root2 == null)
        return false;

    // data mismatch
    if (root1.data != root2.data)
        return false;

    // check mirror condition
    return areMirror(root1.left, root2.right) &&
           areMirror(root1.right, root2.left);
}
```

Problem 26 Give an algorithm for finding LCA (Least Common Ancestor) of two nodes in a Binary Tree.

Solution: Traverse the tree once, if the current node matches n_1 or n_2 , return 1. Check left and Right Subtrees if Both Sides return non-null, current node is the LCA

Algorithm

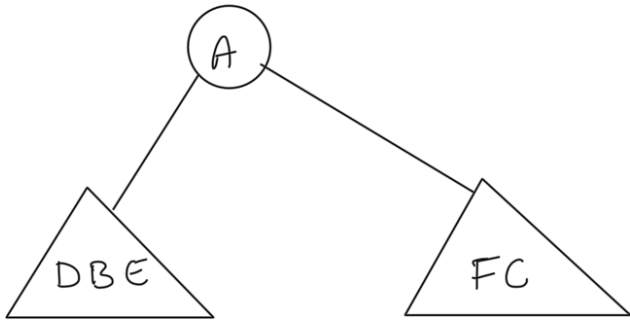
1. if $root == null$, return null
2. if $root.data == n_1$ or $root.data == n_2$, return root
3. Recursively find LCA in
 - left Subtree
 - Right Subtree
4. if Both Recursive Call return non-null
 - Current root is the LCA $\rightarrow$ return root
5. Else
 - Return the non-null value (if any)
6. if Both are null return null

code

```
public static TreeNode lca(TreeNode root, int n1, int n2) {  
    if (root == null)  
        return null;  
  
    // if either n1 or n2 matches root  
    if (root.data == n1 || root.data == n2)  
        return root;  
  
    TreeNode leftLCA = lca(root.left, n1, n2);  
    TreeNode rightLCA = lca(root.right, n1, n2);  
  
    // if both sides return non-null  
    if (leftLCA != null && rightLCA != null)  
        return root;  
  
    // else return the non-null child  
    return (leftLCA != null) ? leftLCA : rightLCA;  
}
```

Problem 27 Give an algorithm for constructing Binary tree from given Inorder and Preorder traversals

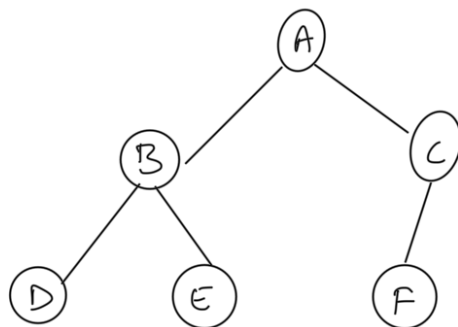
Solution: Let us consider the traversals Below
InOrder Sequence D B E A F C
PreOrder Sequence A B D E C F



In a pre Order Sequence leftmost elements denotes the root of the tree. So we know 'A' is the root for given sequences. By searching 'A' in Inorder Sequence we can find out all elements on the left side of 'A',

Which come under the left subtree, and elements on the right side of 'A', which comes under the right subtree. So we get the structure as seen below.

We recursively follow the above steps and get the following tree



Algorithm :

1. Select an element from PreOrder. Increment a preOrder index Variable (preOrderIndex in code below) to pick next element in the next recursive call.
2. Create a new tree node (newNode) from heap with the data as Selected element.
3. Find the Selected element's index in InOrder. Let the index be inOrderIndex
4. Call Build Binary Tree for elements before inOrderIndex and make the Built tree as left subtree of newNode
5. Call Build Binary Tree for elements after inOrderIndex and make the Built tree as right subtree of newNode.
6. Return newNode.

code

```
public class BuildTree {

    static int preIndex = 0;

    public static TreeNode buildTree(int[] preorder, int[] inorder,
                                     int inStart, int inEnd) {

        // base case
        if (inStart > inEnd)
            return null;

        // pick current node from preorder
        int rootVal = preorder[preIndex++];
        TreeNode root = new TreeNode(rootVal);

        // if only one node
        if (inStart == inEnd)
            return root;

        // find root index in inorder
        int inIndex = search(inorder, inStart, inEnd, rootVal);

        // build left and right subtrees
        root.left = buildTree(preorder, inorder, inStart, inIndex - 1);
        root.right = buildTree(preorder, inorder, inIndex + 1, inEnd);

        return root;
    }

    // helper function to find index in inorder
    private static int search(int[] inorder, int start, int end, int value) {
        for (int i = start; i <= end; i++) {
            if (inorder[i] == value)
                return i;
        }
        return -1;
    }
}
```